

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Bases de Datos 2

PROYECTO 1: MANUAL DE USUARIO

Integrantes:

Harold Steven Catún Guevara - 202145353

Jennifer Yulissa Lourdes Taperio Manuel - 202103763

Fecha: 10 de Junio 2026

Curso: Bases de Datos 2

Manual de Usuario

1. Introducción

Este manual proporciona una guía paso a paso para utilizar el sistema bancario BancoDB. Incluye los detalles para probar los 9 procedimientos almacenados obligatorios y las instrucciones para realizar procesos de recuperación de la base de datos mediante respaldos.

2. Guía de Procedimientos Almacenados (Flujo de Uso)

A continuación, se detalla el flujo lógico de uso del sistema. Usted puede ejecutar estas sentencias consecutivamente para probar todas las operaciones bancarias.

2.1 SP_RegistrarCliente (Paso 1)

- **Descripción:** Ingresa al sistema un nuevo cliente.
- **Uso / Parámetros:** CALL SP_RegistrarCliente(ClienteDNI, Nombre, FechaNacimiento, Direccion, Telefono, Correo);
- **Resultado esperado:** Agrega el nuevo ente al catálogo.

Ejemplo a ejecutar:

```
CALL SP_RegistrarCliente(987654321, 'Juan Pérez', '1990-05-15', 'Ciudad', '55555555',  
'juan@correo.com');
```

```
[2026-06-10 23:50:37] Connected  
[2026-06-10 23:50:37] > USE BancoDB  
[2026-06-10 23:50:37] completed in 2 ms  
[2026-06-10 23:50:37] bancodb> CALL SP_RegistrarCliente(987654321, 'Juan Pérez', '1990-05-15', 'Ciudad Guatemala', '55555555', 'juan@correo.com')  
[2026-06-10 23:50:37] completed in 5 ms
```

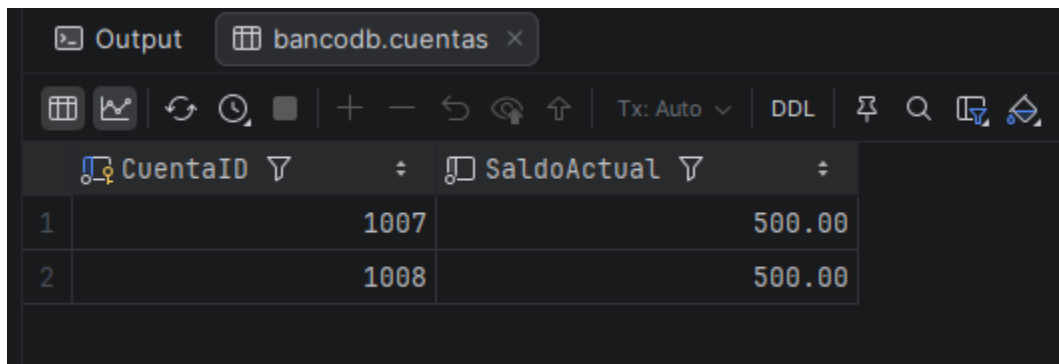
2.2 SP_AbrirCuenta (Paso 2)

- **Descripción:** Registra una nueva cuenta para el cliente que acabamos de crear.
- **Uso / Parámetros:** CALL SP_AbrirCuenta(ClienteDNI, TipoCuenta, SaldoInicial, SucursalID);
- **Resultado esperado:** Se crea en el sistema la cuenta y su estado es 'ACTIVA'. (Revise el ID generado con SELECT * FROM Cuentas; para los siguientes pasos, usualmente es el último o el ID 1).

Ejemplo a ejecutar:

```
CALL SP_AbrirCuenta(987654321, 'Ahorro', 500.00, 1);
```

```
[2026-06-10 23:51:18] bancodb> CALL SP_AbrirCuenta(987654321, 'Ahorro', 500.00, 1)
[2026-06-10 23:51:18] completed in 4 ms
```



	CuentaID	SaldoActual
1	1007	500.00
2	1008	500.00

2.3 SP_RegistrarDeposito (Paso 3)

- **Descripción:** Ingresa dinero a la cuenta actualizando su saldo y registrando la transacción.
- **Uso / Parámetros:** CALL SP_RegistrarDeposito(CuentaID, Monto, Descripcion, EmpleadoID);
- **Resultado esperado:** El saldo total del cliente aumenta según el monto otorgado.

Ejemplo a ejecutar:

```
CALL SP_RegistrarDeposito(1007, 150.00, 'Depósitos de efectivo prueba', 501);
```

```
[2026-06-11 00:01:58] bancodb> CALL SP_RegistrarDeposito(1007, 150.00, 'Depósitos en efectivo', 501)
[2026-06-11 00:01:59] 1 row retrieved starting from 1 in 343 ms (execution: 7 ms, fetching: 336 ms)
```

2.4 SP_RegistrarRetiro (Paso 4)

- **Descripción:** Resta el dinero de una cuenta válida, cuidando que exista el saldo.
- **Uso / Parámetros:** CALL SP_RegistrarRetiro(CuentaID, Monto, Descripcion, EmpleadoID);
- **Resultado esperado:** El dinero disminuye.

Ejemplo a ejecutar:

```
CALL SP_RegistrarRetiro(1007, 50.00, 'Retiro de cajero', 501);
```

```
[2026-06-11 00:03:54] bancodb> CALL SP_RegistrarRetiro(1007, 50.00, 'Retiro de cajero', 501)
[2026-06-11 00:03:54] completed in 5 ms
[2026-06-11 00:03:58] bancodb> CALL SP_RegistrarRetiro(1007, 50.00, 'Retiro de cajero', 501)
[2026-06-11 00:03:58] completed in 4 ms
```

2.5 SP_RealizarTransferencia (Paso 5)

- **Descripción:** Mueve fondos de una cuenta a otra, manejando control de errores (rollback).
- **Uso / Parámetros:** CALL SP_RealizarTransferencia(CuentaOrigenID, CuentaDestinoID, Monto);
- **Resultado esperado:** A la cuenta origen se le descuentan los fondos y a la destino se le aumentan. (Requiere tener al menos dos cuentas creadas).

Ejemplo a ejecutar:

```
CALL SP_RealizarTransferencia(1007, 1008, 200.00);
```

Output Result 11

```
[2026-06-11 00:05:06] bancodb> CALL SP_RealizarTransferencia(1007, 1008, 200.00)
[2026-06-11 00:05:06] 1 row retrieved starting from 1 in 341 ms (execution: 5 ms, fetching: 336 ms)
```

2.6 SP_ReporteClientesSucursal (Paso 6)

- **Descripción:** Muestra un listado de clientes por nombre de sucursal.

- **Uso / Parámetros:** CALL SP_ReporteClientesSucursal(NombreSucursal);
- **Resultado esperado:** Una vista tabular limpia que expone únicamente los requeridos.

Ejemplo a ejecutar:

```
CALL SP_ReporteClientesSucursal('Sucursal Central');
```

```
[2026-06-11 00:05:45] bancodb> CALL SP_ReporteClientesSucursal('Sucursal Central')
[2026-06-11 00:05:46] 0 rows retrieved in 327 ms (execution: 6 ms, fetching: 321 ms)
```

2.7 SP_ActualizarCliente (Paso 7)

- **Descripción:** Actualiza campos de contacto en un registro de cliente ya existente.
- **Uso / Parámetros:** CALL SP_ActualizarCliente(ClienteDNI, Direccion, Telefono, Correo);
- **Resultado esperado:** Modifica dirección y teléfono.

Ejemplo a ejecutar:

```
CALL SP_ActualizarCliente(987654321, 'Nueva Dirección', '66666666',
'jperez@correo.com');
```

```
[2026-06-11 00:08:16] bancodb> CALL SP_ActualizarCliente(987654321, 'Nueva Dirección', '66666666', 'jperez@correo.com')
[2026-06-11 00:08:16] completed in 4 ms
```

2.8 SP_ReporteMovimientosCuenta (Paso 8)

- **Descripción:** Recupera el historial completo de movimientos para una cuenta en un marco de tiempo.
- **Uso / Parámetros:** CALL SP_ReporteMovimientosCuenta(CuentaID, FechaInicio, FechaFin);
- **Resultado esperado:** Devuelve el grid con los movimientos (incluyendo depósitos, retiros y transferencias previas).

Ejemplo a ejecutar:

```
CALL SP_ReporteMovimientosCuenta(1007, '2026-06-01', '2026-06-30');
```

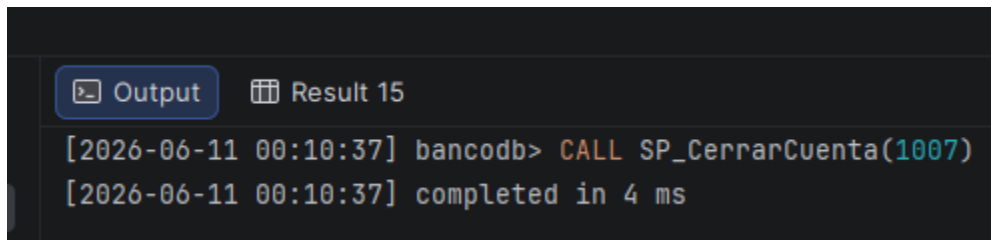
Output Result 15				
	Fecha	Descripcion	Monto	TipoTransaccion
1	2026-06-11 00:01:58	Depósitos en efectivo	150.00	DEPOSITO
2	2026-06-11 00:03:54	Retiro de cajero	50.00	RETIRO
3	2026-06-11 00:03:58	Retiro de cajero	50.00	RETIRO
4	2026-06-11 00:05:06	Transferencia enviada	200.00	TRANSFERENCIA_SALIDA

2.9 SP_CerrarCuenta (Paso 9)

- **Descripción:** Desactiva o cierra una cuenta. Exige que el saldo sea 0 para proceder.
- **Uso / Parámetros:** CALL SP_CerrarCuenta(CuentaID);
- **Resultado esperado:** El estado de cuenta cambia a 'CERRADA'. (Si esta cuenta probada aún tiene dinero tras las transferencias, realice retiros de prueba hasta llegar a Q0.00 antes de correr esto).

Ejemplo a ejecutar:

```
CALL SP_CerrarCuenta(1007);
```



The screenshot shows a terminal window with a dark background. At the top, there are two tabs: 'Output' and 'Result 15'. The 'Output' tab is active. Below the tabs, the terminal displays the command `CALL SP_CerrarCuenta(1007)` and the response `completed in 4 ms`. The timestamp `[2026-06-11 00:10:37]` is visible on both lines.

3. Guía Paso a Paso para la Ejecución del Proyecto (Despliegue y Recuperación)

3.1 Prerrequisitos e Instalación

1. Instalar las dependencias de Python necesarias abriendo una terminal:

```
pip check
pip install python-dotenv sqlalchemy pymysql
```

2. Asegurarse de tener instalado y configurado mysqldump en las variables de entorno de su sistema operativo (comprobar con el comando `mysqldump --version`).
3. Clonar o cambiar el nombre del archivo `.env.example` dejándolo como `.env` y configurar ahí las variables y credenciales de acceso a MySQL.

Para solventar errores de colisión por procedimientos existentes u olvido de contexto, recomendamos cargar todo usando un script consolidado que incluya encabezados de borrado:

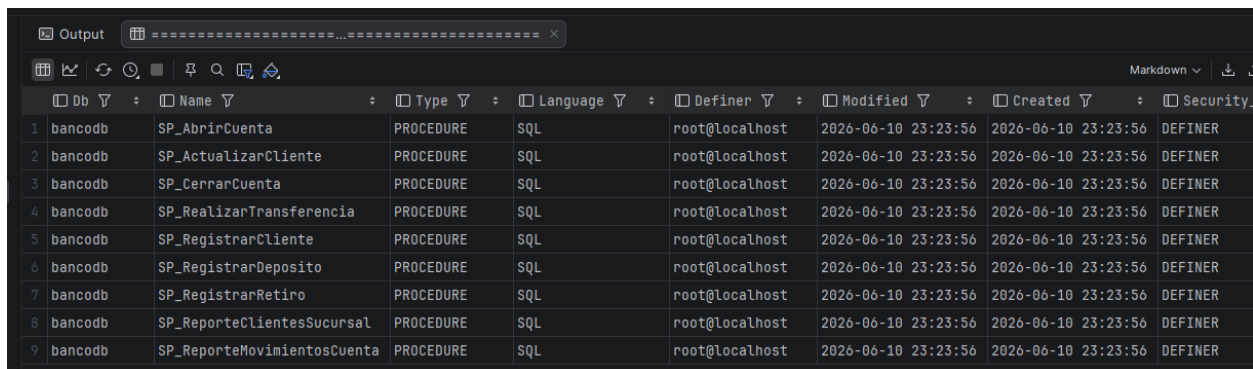
4. Ejecutar `1_database.sql` (crea la estructura de tablas base).

5. Ejecutar de ser posible el archivo que contenga todo consolidado, anteponiendo explícitamente:

```
USE BancoDB;  
DROP PROCEDURE IF EXISTS ... -- (Limpiar TODOS los existentes primeros)
```

Si se usan los archivos incrementales individuales (2_incremental...), asegúrese de rodearlos igual con el USE BancoDB;

6. Ejecutar 4_users.sql para crear los roles e impartir los GRANTS.

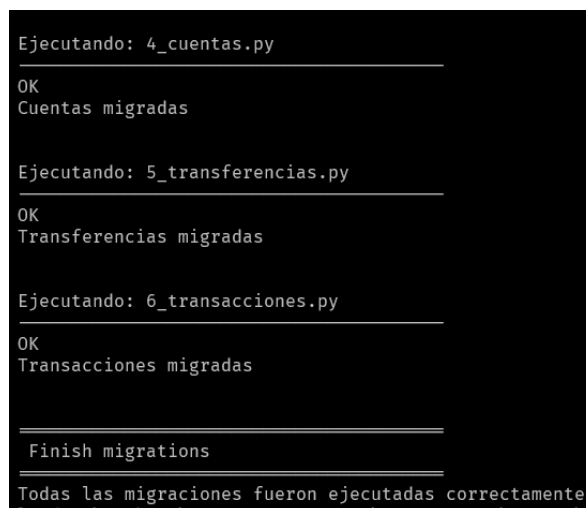


	Db	Name	Type	Language	Definer	Modified	Created	Security
1	bancodb	SP_AbrirCuenta	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
2	bancodb	SP_ActualizarCliente	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
3	bancodb	SP_CerrarCuenta	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
4	bancodb	SP_RealizarTransferencia	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
5	bancodb	SP_RegistrarCliente	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
6	bancodb	SP_RegistrarDeposito	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
7	bancodb	SP_RegistrarRetiro	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
8	bancodb	SP_ReporteClientesSucursal	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER
9	bancodb	SP_ReporteMovimientosCuenta	PROCEDURE	SQL	root@localhost	2026-06-10 23:23:56	2026-06-10 23:23:56	DEFINER

3.3 Migración y Limpieza de Datos (Python)

Para realizar el ETL desde los archivos CSV, diríjase a la carpeta /python en terminal y ejecute:

7. python reset.py: Limpia la base de datos eliminando registros anteriores si los hubiera.
8. python run.py: Orquesta y ejecuta la migración masiva de datos en el orden apropiado (sucursales, clientes, empleados, cuentas, etc.) asegurando la integridad referencial de las llaves foráneas.



```
Ejecutando: 4_cuentas.py  
OK  
Cuentas migradas  
  
Ejecutando: 5_transferencias.py  
OK  
Transferencias migradas  
  
Ejecutando: 6_transacciones.py  
OK  
Transacciones migradas  
  
Finish migrations  
  
Todas las migraciones fueron ejecutadas correctamente.
```

3.4 Creación de Respaldos (Backups)

Para la estrategia de seguridad, en la raíz del proyecto se incluye el script de volcado:

- Ejecutar mediante la terminal:

```
python full_backup.py
```

- Este comando creará la carpeta /backups y generará un respaldo completo (ej. respaldo_comp_[fecha].sql). Está diseñado para hacer respaldo "full" de estructuras y tablas limpias, dejando por aparte los procedimientos que se recuperan mediante los archivos incrementales.
- Si se desea automatizar, puede añadirse a las Tareas Programadas de Windows (Task Scheduler).

```
> python full_backup.py
```

```
RESPALDO COMPLETO
```

3.5 Pruebas Computacionales ACID (Pruebas de Concurrencia)

Para las validaciones de concurrencia y prevención de anomalías, en la carpeta /tests_acid se incluyen tests listos para ejecutarse:

- python test_deadlock.py: Prueba cómo responde el sistema ante bloqueos mutuos simulando dos cuentas.

```
> python test_deadlock.py
¿Probar el deadlock? (s/n): S
Sesion 2: Bloqueando cuenta 1002
Sesion 1: Bloqueando cuenta 1001
Sesion 2: Intentando bloquear cuenta 1001
Sesion 1: Intentando bloquear cuenta 1002
Sesion 1 ERROR:
(pymysql.err.OperationalError) (1213, 'Deadlock found when trying to get lock; try restarting transaction')
[SQL:
        SELECT *
        FROM Cuentas
        WHERE CuentaID = 1002
        FOR UPDATE
    ]
(Background on this error at: https://sqlalche.me/e/20/e3q8)
C:\Users\dakot\Desktop\AAAsemetre2026-01\VACCACIONES\BASES2\laboratorioG7\BD2_VACJUN26_Proyectos_G7\Primer Proyecto\test
s_acid\test_deadlock.py:102: SAWarning: transaction already deassociated from connection
  trans.rollback()
Prueba terminada
```


- python test_transferenciaConcurrente.py: Simula transacciones ejecutándose en paralelo.

```

lurde ~\Desktop\AAAsemetre2026-01\VACCACIONES\BASES2\laborat
¿Ejecutar 20 transferencias simultáneas? (s/n): s

=====
PRUEBA DE CONCURRENCIA
=====
Saldo total inicial: 4700.00
[OK] Transferencia 1
[OK] Transferencia 6
[OK] Transferencia 11
[OK] Transferencia 3
[OK] Transferencia 2
[OK] Transferencia 4
[OK] Transferencia 15
[OK] Transferencia 19
[OK] Transferencia 16
[OK] Transferencia 17
[OK] Transferencia 18
[OK] Transferencia 8
[OK] Transferencia 5
[OK] Transferencia 20
[OK] Transferencia 13
[OK] Transferencia 10
[OK] Transferencia 7
[OK] Transferencia 9
[OK] Transferencia 14
[OK] Transferencia 12

=====
RESULTADOS
=====
Transferencias exitosas : 20
Errores encontrados    : 0
Saldo inicial : 4700.00
Saldo final   : 4700.00

[OK] No se perdió dinero
Tiempo total: 0.18 segundos

```

4. Resolución de Problemas Comunes (Troubleshooting)

Problema	Causa Probable	Solución
Transferencias arrojan ROLLBACK intencionalmente	Se intentó hacer una transferencia por Q300 en una cuenta que solo tenía Q150, o el DNI origen no existe.	Verifica tus datos haciendo unas lecturas SELECT directas sobre la cuenta o intentando montos a tono al Saldo, los procedimientos bloquean la trampa a propósito para resguardar saldos íntegros.